

Specifica e Verifica di Sistemi Critici



Elvinia Riccobene
elvinia.riccobene@unimi.it

Motivazioni



Perche' e cosa studieremo

Sistemi Software-dependent



- air traffic control systems,
- medical devices,
- automotive systems,
- financial systems
- Telecommunications and IoT services
- ...

Sistemi Software-dependent

La nostra vita quotidiana e le attività umane dipendono dai computer...

- dai *beni finali* (orologi, dispositivi elettronici, telefoni, automobili, ecc.)
- alle più *grandi infrastrutture* (energia, trasporto, ecc.),
 - molte funzioni che in precedenza venivano eseguite meccanicamente o elettricamente sono ora gestite digitalmente

Sistemi Software-dependent

Questo fenomeno è stato reso possibile da una combinazione di fattori:

- Aumento della potenza di calcolo
- Aumento delle capacità di archiviazione dei dati
- Aumento della connettività
- Aumento della produttività del software

Pros e cons su computer automation

(+) offre dispositivi e infrastrutture più flessibili e performanti

(-) aumenta il rischio di guasti o malfunzionamenti ed esposizione ad attacchi

Sistemi critici per sw automation

Due classi di sistemi per cui il rischio può avere conseguenze drammatiche:

- **Life-critical systems** (detti anche *safety-critical systems*)
 - guasti o malfunzionamenti mettono in pericolo la vita umana
- **Mission-critical systems** (detti anche *business-critical systems*)
 - guasti o malfunzionamenti possono generare perdite finanziarie
 - **Security-critical systems**

Esempi di fallimento di sistemi

- https://en.wikipedia.org/wiki/List_of_software_bugs
- Safety-critical systems:
 - Therac 25 radiotherapy machine
 - Medtronic heart device
 - Hole in the Ozone
 - ecc
- Business-critical systems
 - Ariane 5 rocket explosion
 - Mars Climate Orbiter Catastrophe
 - Denver airport automated baggage system (1994)
 - ecc

10 historical software bugs

Deadly Radiation therapy

Many people were exposed to deadly amounts of radiation between 1985 and 1997 because the Therac-25 device that was being used to treat them had faulty software. As a result, a number of people died due to excessive exposure to radiation.



10 historical software bugs

▣ Medtronic heart device

- Medtronic this week alerted health care providers to a defect in its Heartware Ventricular Assist Device System that may cause the internal pump to malfunction



FDA: Medtronic heart device could malfunction

© Apr 29, 2022 - 04:03 PM

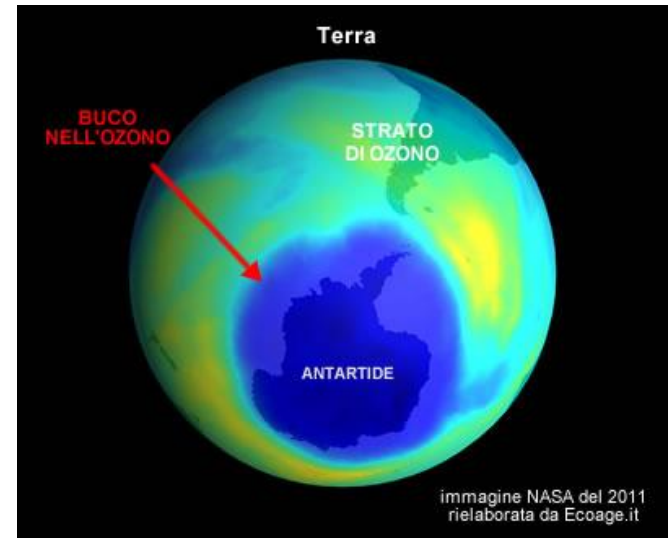


Medtronic this week alerted health care providers to a defect in its Heartware Ventricular Assist Device System that may cause the internal pump to malfunction, the Food and Drug Administration [announced](#) yesterday. The company is investigating to determine which devices may be affected. FDA recommends that health care providers and patients make decisions about removing or exchanging the device on a case-by-case basis, considering the patient's clinical status and surgical risks.

10 historical software bugs

Hole in the Ozone

The hole that was in the ozone layer over the Antarctica remained undetected for a long time, largely due to the fact that the software that was being used by NASA to monitor the ozone had been programmed to ignore values that deviated greatly from expected measurements.



10 historical software bugs

Ariane 5 Explosion

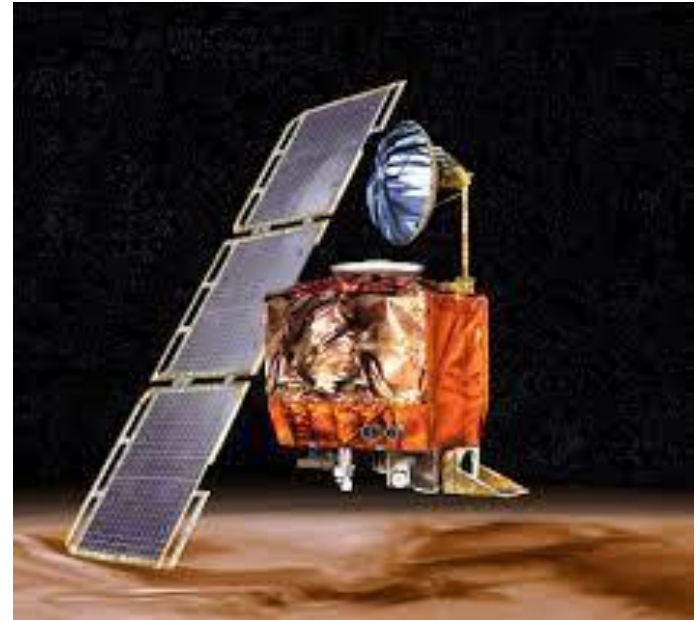
\$370 million were lost in 1996 when the Ariane 5 rocket exploded due to the fact that software from the Ariane 4 had been used and had not been configured for use on faster engines.



10 historical software bugs

The Mars Climate Orbiter Catastrophe

One of the subcontractors NASA used when building its Mars climate orbiter had used English units instead of the intended metric system, which caused the orbiter's thrusters to work incorrectly. Due to this bug, the orbiter crashed almost immediately when it arrived at Mars in 1999.



Window web security bugs

Patch distribuito da Microsoft a febbraio 2026, correggendo **59 vulnerabilità**:

▣ 5 sono classificate come *critiche*, 52 come *importanti* e 2 come *moderate*:

- 25 di tipo escalation di privilegi;
- 12 di esecuzione di codice remoto;
- 7 di tipo spoofing;
- 6 di tipo Information Disclosure;
- 5 di bypass delle funzionalità di sicurezza;
- 3 di tipo Denial of Service;
- 1 di tipo cross-site scripting.

Cause di fallimenti e malfunzionamenti

- ❑ Errori hardware
- ❑ Bug del software
- ❑ Problemi di sicurezza
- ❑ Problemi di prestazioni
- ❑ Uso di AI ([How secure is AI-generated Code](#))
- ❑ **Errori di progettazione**
 - Spesso la causa principale

Problemi che influenzano la corretta progettazione

- Riduzione dei costi
- Riduzione del time-to-market
- Nuove funzionalità aggiunte (a sistemi esistenti) per meglio soddisfare le esigenze del cliente
- Complessità dei sistemi
 - Aspetti di Fault-tolerant
 - Concorrenza
 - Utilizzo interattivo di più piattaforme
 - Gestione delle legacy applications
 - Algoritmi di ottimizzazione
 - Autonomia dei sistemi ed incertezza
 - Necessità di sicurezza

Come garantire la correttezza di funzionalità di un sistema?

- Problema identificato già alla fine degli anni 60
- Approcci differenti al problema:
 - **Approcci organizzativi:** considerano il problema come un'istanza particolare del problema più generale della qualità del prodotto
 - Varie metodologie e standard: **ISO 9001** (Quality management systems – Requirements), **CMMI** (Capability Maturity Model Integration), **ISO 15504** (Software Process Improvement and Capability dEtermination).
 - **Approcci tecnici:** Basati sul controllo del software
 - Principalmente su **testing** in grado di rilevare la maggior parte degli errori
 - Errori residui (detti *high-quality bugs*) rimangono inosservati e sono quelli più critici per i sistemi life-critical o mission-critical

Metodi formali

Considerati *the best candidates* per lo sviluppo *a zero difetti* di software-dependent systems

I metodi formali sono tecniche matematicamente ben fondate progettate per assistere lo sviluppo di sistemi complessi basati su computer

The White House Makes It Clear (Feb 2024):
“While formal methods have been studied for decades, their deployment remains limited; further innovation in approaches...is vital to accelerate broad adoption.”

Metodi formali nel processo di sviluppo

Scopo dei FMs:

- **Design time**: per la costruzione di sistemi a zero difetti
- **Development time**: per la ricerca di difetti nella fase di codifica ed integrazione
- **Run time**: per stabilire che i sistemi in funzione sono a zero difetti

Tre caratteristiche principali

Linguaggi:

- Notazioni matematiche con semantica formale utilizzate per specificare modelli e proprietà attese
 - I sistemi HW/SW sono considerati come *oggetti matematici* che possono essere descritti e analizzati in modo rigoroso

Tools:

- Strumenti SW che possono assistere la progettazione durante il processo di sviluppo

Metodologie:

- *Linee guida* per un uso corretto di un FM nello sviluppo di sistemi reali
- *Processo* per supportarne l'uso a design/development/run time
- *Proof strategies*

Modello o specifica

- È la **descrizione di un sistema in termini di oggetti matematici** (utilizzando la notazione rigorosa dei FMs)
 - questo non significa che l'utente deve essere un matematico!!!

- **Modellare implica astrarre**
 - Semplificare la descrizione del sistema, conservando solo alcuni di dettagli originali, e focalizzandosi sulle proprietà principali

Vantaggi dei metodi formali

- Le specifiche in linguaggio naturale sono:
 - inconsistenti
 - ambigue
 - difficili da seguire
- Le specifiche formali
 - forzano a pensare più a fondo sul significato della specifica scritta
 - possono essere usate per provare la correttezza del programma
 - possono essere usate per generare casi di test

Limiti dei metodi formali

- **scrivere una specifica formale** è come scrivere un programma:
 - è possibile introdurre errori,
 - non si è comunque sicuri di catturare il comportamento “inteso” del programma,
- fare **prove di correttezza** è un processo lungo e difficile:
 - su carta, è facile introdurre errori,
 - su calcolatore, chi assicura che (il programma usato) sia corretto?

Diversi formalismi

□ Formalismi operazionali:

- forniscono una descrizione del comportamento del sistema in termini di operazioni di una macchina astratta

□ Formalismi dichiarativi:

- definiscono il sistema in termini di proprietà che devono essere soddisfatte

Diversi paradigmi computazionali

Specifica operativa:

Definisce *cosa* dovrebbe fare un sistema o un componente e definisce (almeno in parte) *come* dovrebbe farlo

- *I modelli sono costruttivi*, il che significa che è possibile utilizzarli per generare automaticamente un'implementazione del sistema o del componente (o, almeno, uno scheletro di tale implementazione).

Specifica dichiarativa:

Definisce *cosa* dovrebbe fare un sistema o un componente, ma non *come* dovrebbe farlo

- Esprime obiettivi e vincoli (*proprietà*) che qualsiasi corretta implementazione del sistema o del componente dovrebbe soddisfare
- *I modelli sono non costruttivi*, nel senso che sarebbe impossibile (o, almeno, molto difficile) derivare automaticamente da queste proprietà un'implementazione efficiente del sistema o del componente

Per comprendere la differenza

- *Esempio 1:* Consideriamo un programma per l'ordinamento di una matrice di numeri interi.
 - Una specifica dichiarativa indicherà che il programma termina e che, dopo la sua terminazione, l'array è ben ordinato.
 - Una specifica operativa fornirà un algoritmo di ordinamento o una classe di algoritmi di ordinamento.

- *Esempio 2:* Consideriamo un protocollo di comunicazione su un canale inaffidabile.
 - Una specifica dichiarativa richiederà che il protocollo trasmetta correttamente ogni messaggio.
 - Una specifica operativa descriverà in dettaglio come il protocollo rileva le perdite di messaggi ed esegue le ritrasmissioni quando necessario.

Cosa si intende per analisi

Definizioni dallo standard IEEE Sd. 1012-1986:

Validazione

- il processo di valutare un sistema o una componente durante o alla fine del processo di sviluppo per determinare **se** esso **soddisfa i requisiti specificati**
 - **Model validation**

Verifica

- il processo di valutare un sistema o una componente per determinare, durante il processo di sviluppo, se i prodotti soddisfano le condizioni (o **proprietà**) richieste
 - **Model verification**

Formalismi operazionali

- Facilitano la **validazione**
- Esempi di formalismi operazioni:
 - Abstract State Machines
 - B method
 - Z method
 - SCR (Software Cost Reduction)
 - Reti di Petri

Formalismi dichiarativi

- Facilitano la **verifica**
- Esempi di formalismi dichiarativi:
 - Logica temporale
 - Algebre dei processi
 - Coq

Strumenti per Verifica e Validazione

- **Validazione**: necessaria per controllare che il sistema soddisfa i requisiti richiesti
 - Attraverso simulazione e testing

- **Verifica**: necessaria per garantire proprietà
 - Attraverso dimostrazioni di proprietà
 - con theorem prover e model checker
 - Non determina errori causati da specifiche incorrette
 - Gli errori possono essere propagati nelle fasi successive

I metodi formali hanno una lunga storia

- Inizialmente :
 - Focus sulla semantica dei programmi
 - Uso della logica matematica per dimostrare la correttezza formale di un programma (es. Tripla di Hoare)
- Nel 1962:
 - Le Reti di Petri vengono usate per formalizzare sistemi concorrenti
- Usati per verifica di hardware prima che per quella del software
- Oggi usati per garantire l'affidabilità e se possibile la correttezza del sw ed integrati nel processo di sviluppo
- Nuovi domini applicative e nuove sfide: computer security, hybrid systems/cyberphysics, multi-agent systems, bioinformatics, self-adapting system

Impatto aziendale limitato

I FM non sono abitualmente utilizzati nell'industria, con la notevole eccezione di due classi di domini applicativi, in cui i metodi formali svolgono un ruolo significativo:

- *mission-critical systems*

- per i quali le società di progettazione assumono esperti di metodi formali e utilizzano strumenti di verifica formale (e.g., model checkers and/or theorem provers) nell'ambito del loro processo industriale

- *life-critical systems*

- per i quali i metodi formali sono legalmente richiesti dagli standard o enti di certificazione (es. FDA, CE, ecc)

Modelli di sviluppo per software safety-critical

7 Software RISK MANAGEMENT

5.1
Software
development
planning

5.2
Software
requirements
analysis

5.3
Software
ARCHITECTURAL
design

5.4
Software
detailed
design

5.5
Software UNIT
implementation and
VERIFICATION

5.6
Software integration
and integration
testing

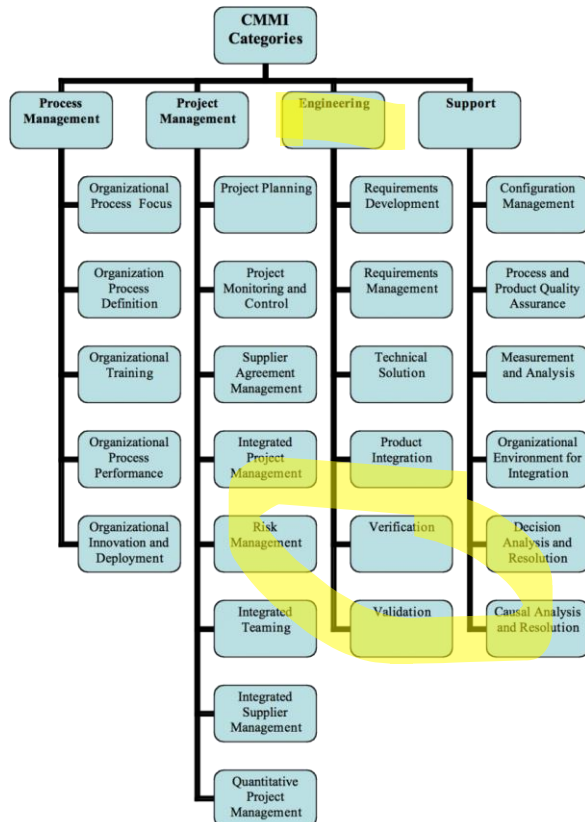
5.7
Software SYSTEM
testing

5.8
Software release

8 Software configuration management

IEC 62304 standard per sw medico

Modelli di sviluppo per software security-critical



- Capability Maturity Model Integration (CMMI)
- CMMI-DEV(elopment) Process Areas

Alcuni preconcetti

- I metodi formali posso essere usati solo dai matematici
 - Errato! Hanno un rigoroso fondamento matematico ma l'utente dovrebbe non preoccuparsene
- Il processo di verifica è esso stesso soggetto ad errori, quindi perchè preoccuparsene?
 - Lo adottiamo per ridurre gli errori, non per eliminarli completamente
- L'uso di metodi formali rallenta la realizzazione del progetto
 - è probabile che invece lo velocizzi, dal momento che gli errori sono scoperti nelle fasi iniziali

In questo scenario cosa studieremo?

Finalità del corso

- Il corso presenta le fondamentali tecniche per la modellazione e la verifica formale di sistemi *software-dependent*
- Impareremo:
 - i **linguaggi di specifica**
 - che consentono di descrivere un sistema da analizzare e le proprietà da provare
 - i fondamenti teorici delle **metodologie di analisi** (validazione e verifica) basate su simulazione, costruzione di scenari e model checking
 - gli **strumenti automatici** (tool) che consentono l'analisi formale

Il nostro approccio

- **Imparare vari metodi** (la verifica deduttiva, il model checking, il testing da specifiche).
- **Imparare vantaggi e limiti** allo scopo di scegliere i metodi giusti ed i tool più opportuni.
- **Imparare come combinare diversi metodi** (ad es.
 - verifica deduttiva e model checking
 - metodi formali e semi-formali)

Enfasi su:

- **Processo** di selezionare ed usare metodi formali
 - modellare, verificare, imparare a localizzare errori

- **Notazione** appropriata
 - ASMs (per specifiche operazionali)
 - Logica temporale (per specifiche dichiarative)

- **Uso di tools**
 - ASMETA, NuSMV

Argomenti del corso (1)

□ Automi

- di Mealy e Moore, estesi, concorrenti (ripasso)
- Abstract State Machines e ASMETA
- di Kripke

□ Logica Temporale

- Linear-Time Temporal Logic (LTL)
- Branching-Time Logic (CTL)

Argomenti del corso (2)

- Model checking
 - Symbolic Model Checking
 - NuSMV
- Specificare proprietà
 - proprietà di raggiungibilità, di safety, di liveness, di fairness, assenza di deadlock

Testi

- E. Börger, R. Stärk. **Abstract State Machines**: A Method for High-Level System Design and Analysis, Springer Verlag, 2003.
 - Michael Huth , Mark Ryan. **Logic in Computer Science: modelling and reasoning about systems** (2nd edition). Cambridge University Press, 2004.
 - B. Berard et al., **System and Software Verification** Model-Checking Techniques and Tools, Springer Verlag, 2001.
- + **ASMETA** (documentazione su myAriel)

Ulteriori informazioni

- Ricevimento: su appuntamento
- Materiale didattico su piattaforma myAriel
- Metodologie d'esame: scritto + prova pratica/progetto

Iniziamo....

Automati per la specifica di sistemi



(ripasso)

Automi a Stati Finiti

- Un **automa** (o **macchina**) a stati finiti, abbreviata FSM (Finite State Machine), è una notazione formale che permette la rappresentazione astratta del comportamento di un sistema
- Le FSM hanno:
 - una rigorosa definizione matematica
 - una intuitiva rappresentazione grafica tramite diagrammi di stato

Automa a stati finiti: definizione

- Una FSM è una tupla (S, I, δ)
 - S : insieme finito di stati
 - I : insieme finito di eventi di input
 - $\delta : S \times I \rightarrow S$: funzione di transizione

FSM: rappresentazione grafica

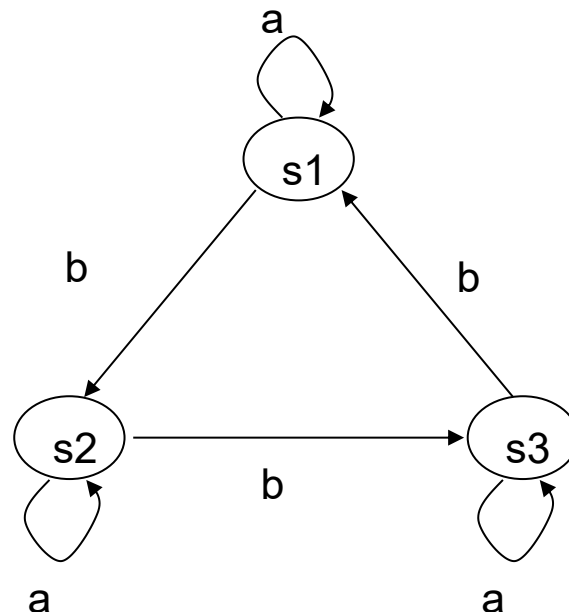
- Un **diagramma di stato** è un grafo direzionato i cui nodi rappresentano gli **stati** ed i cui archi, etichettati dagli input, rappresentano le **transizioni di stato**

Esempio:

$S = \{s1, s2, s3\}$

$I = \{a, b\}$

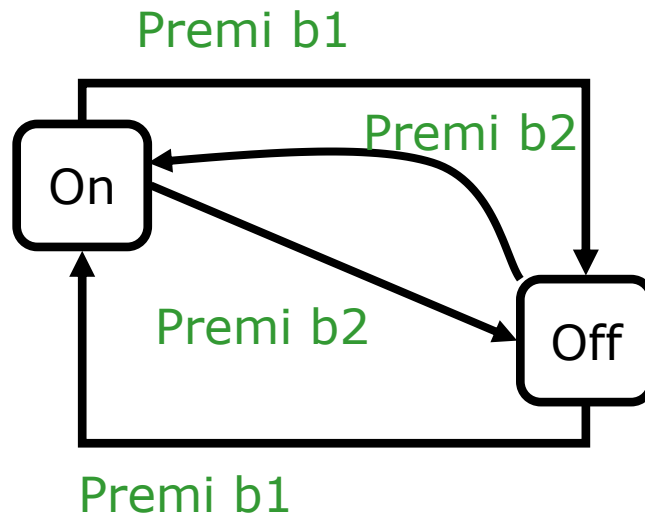
$\delta = \{(s1, a, s1), (s1, b, s2),$
 $(s2, a, s2), (s2, b, s3),$
 $(s3, a, s3), (s3, b, s1)\}$



Modellare con FSMs

- Sono utilizzate per modellare
 - GUIs
 - Protocolli di rete
 - Pacemakers e teller machines
 - Applicazioni WEB
 - Software di sicurezza
 - Sistemi embedded

Esempio: FSM di base



Lampada a 2 interruttori

Modellare con FSMs: limiti

- ❑ Non tutti i requisiti possono essere specificati con una FSM
- ❑ Tra i requisiti non specificabili in FSM:
 - requisiti real time
 - requisiti riguardanti performance
 - requisiti riguardanti tipi di computazioni
- ❑ E' possibile rappresentare solo un numero finito di stati
- ❑ FSM non sono composizionali

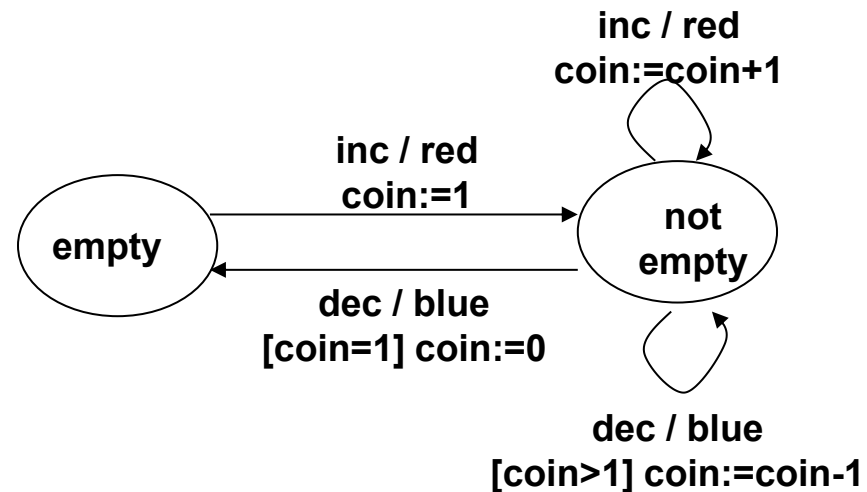
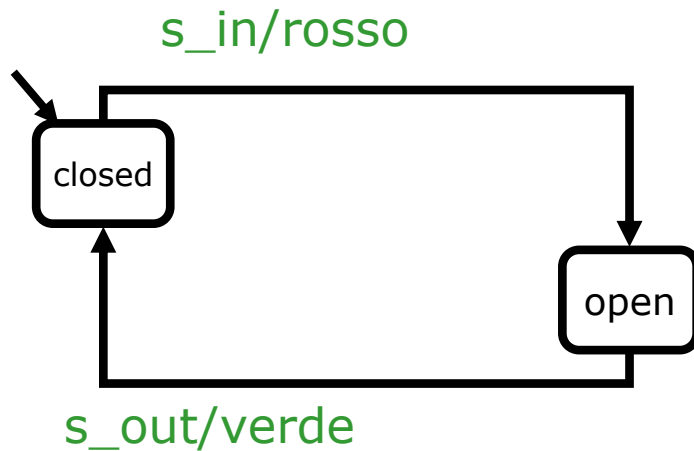
Modellare con FSM

- Il modello FSM di un sistema non è unico
- Modelli diversi in base al livello di **astrazione/dettaglio** considerato
- Lo **stato** è non strutturato
 - non consente di modellare informazioni
- Le **operazioni** sono elementari
 - input esterni
 - output della macchina (automi di Mealy e di Moore)

Estensioni delle FSM (di base)

- Per modellare l' output
 - Automi di Mealy e di Moore
 - **transizione** è una tupla (s, i, o, s')
- Per modellare la memoria della macchina
 - FSM estese
 - **transizione** è una tupla (s, i, o, g, a, s')
- Per modellare la comunicazione
 - FSM di comunicazione
 - coppia (C, P) , C: canali, P: processi (FSM)
 - **transizioni** (s, null, s') , $(s, c?i, s')$, $(s, c!o, s')$

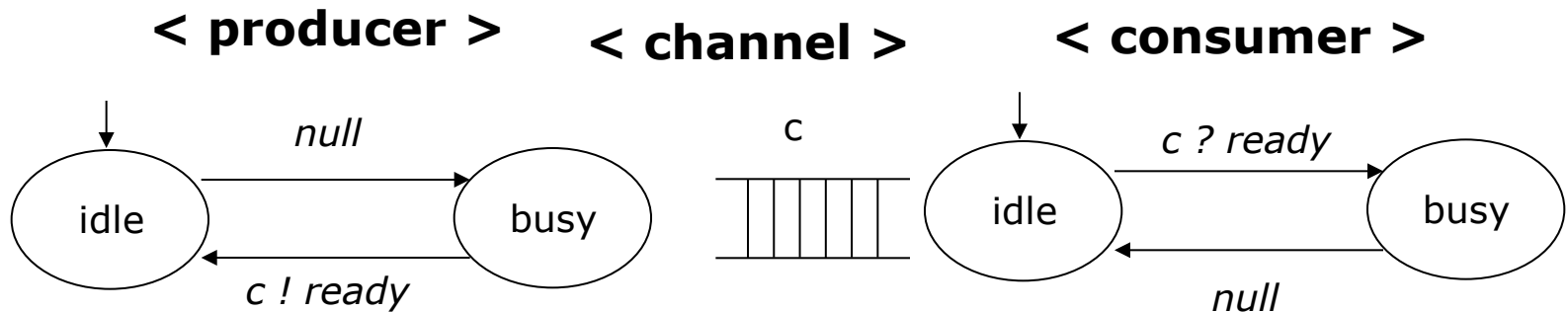
Esempi: FSM di Mealy ed estesa



Sbarra con sensori arrivi auto
e lampada segnalazione

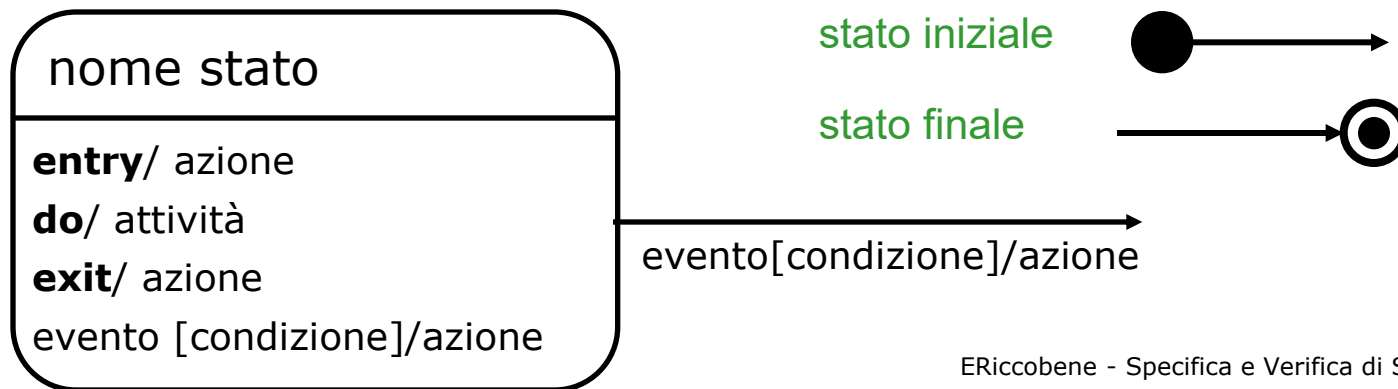
Salvadanaio con luce

Esempio FSM di comunicazione

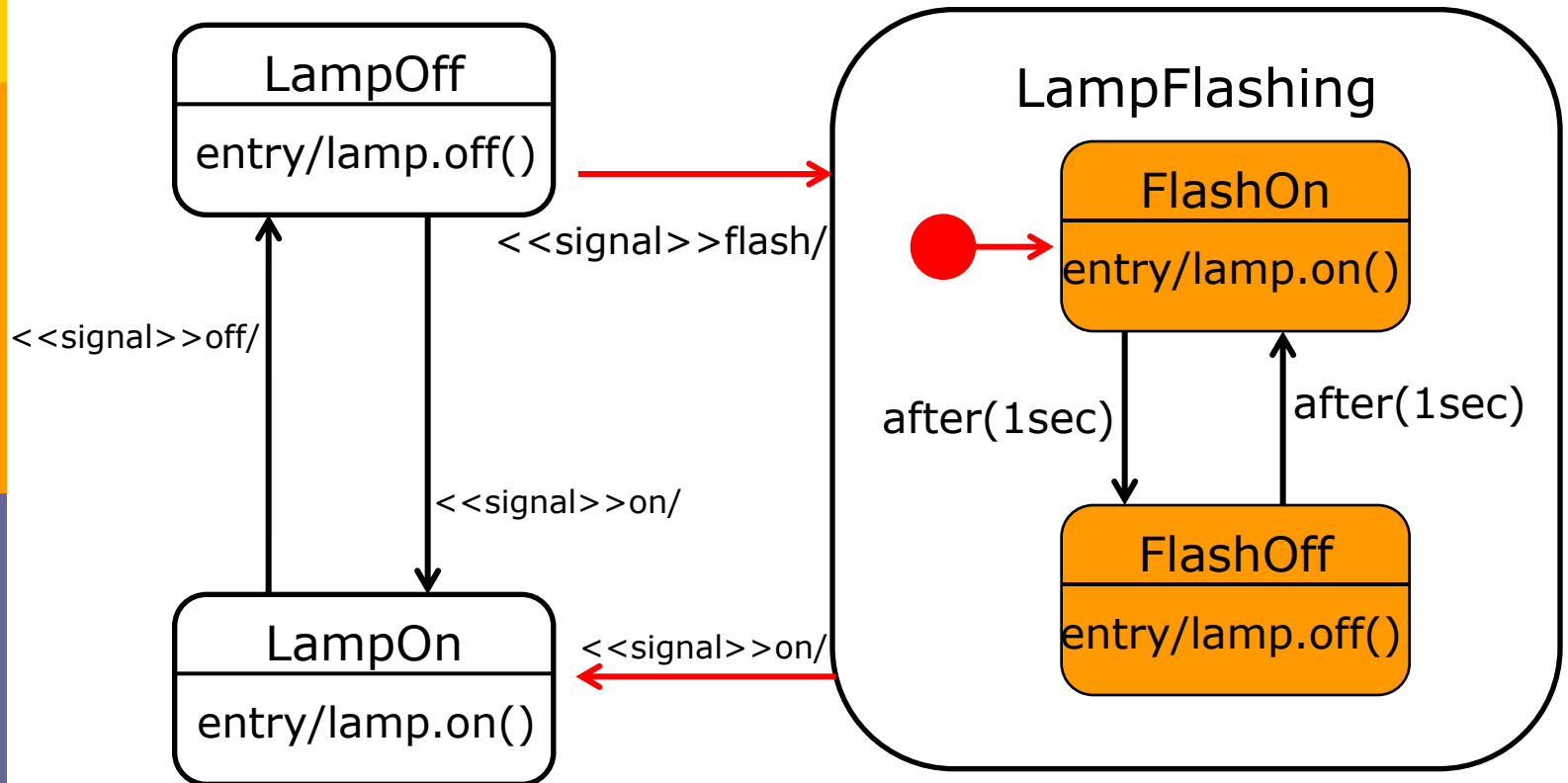


Estensioni delle FSM (di base)

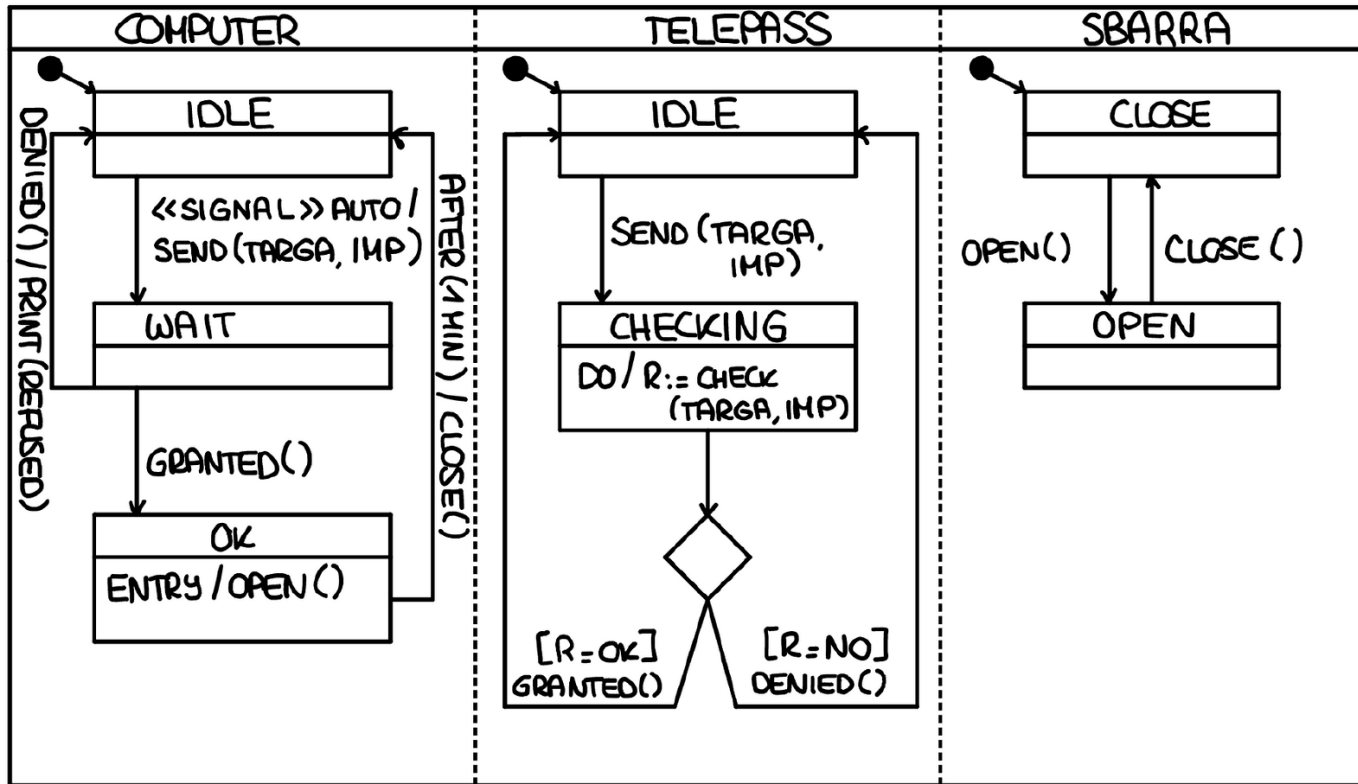
- Per modellare il tempo
 - FSM temporizzate
 - **transizione** è una tupla $(s, g, a|I/O, s', [t1,t2])$
- Per modellare stato e transizioni strutturate, sottomacchina e computazione parallela
 - FSM di Harel (macchine di stato di UML)



UML state machine sequenziale



UML state machine concorrente



Altre estensioni

- Abstract State Machines

- ASM = FSM con stati generalizzati

- Automi di Kripke

- lo stato porta informazione di una proprietà che è valida in esso